ConceptBook: A Graph-First Framework for AI-Generated Curricula

Wen G. Gong
Independent Researcher
wen.gong.research@gmail.com

July 2, 2026

Abstract

ConceptBook treats teaching and learning as two sides of the same coin, unified by a single concept-graph - a directed acyclic graph of primitive concepts, composition relationships, optional per-concept verifiers, and a target mastery concept. *Teaching* is authoring this graph; SPL converts it into a prerequisite-ordered, LLM-generated concept-book. *Learning* is navigating the same graph - following the shortest prerequisite path to reach a chosen target. The concept-graph is the shared currency: one artifact drives both sides.

We apply SPL (a declarative AI workflow language) as the conversion engine and demonstrate two authoring paths. **Path A** *(new concept-book)*: a domain expert authors a concept-graph as a structured file, and the SPL pipeline generates a complete concept-book with zero domain-specific code. **Path B** *(from existing textbooks)*: given an open-source textbook (currently OpenStax, via a pluggable ingester), a separate ingestion pipeline extracts the concept-graph, producing a companion graph-guided concept-book. Both workflows feed the same SPL pipeline; only the source of the concept-graph differs.

The framework is validated across twenty heterogeneous domains, each exhibiting the same *LEGO payoff shape*: a small primitive brick set plus one composition rule generates a substantially larger concept space - a design pattern the framework makes explicit and machine-checkable, and one already documented at full domain scale in prior work on Chinese characters, where 422 elemental characters compose into the ~6,000 characters in common use [Gong 2025]. This is an independent, self-funded research project released fully open source under Apache 2.0; we report the architecture and its structural self-checks, and invite the community to contribute independently-authored domain graphs, content-quality evaluations, and learner studies.

Keywords: educational AI, concept graph, AI-generated curriculum, prerequisite ordering, SPL, open-source textbooks

1 Introduction

1.1 The Missing Layer in Educational AI

The current generation of AI tutoring systems divides into three families. Conversational tutors (Khanmigo [Khan Academy 2023], Socratic) engage learners in dialogue but carry no explicit model of what concepts depend on what. Adaptive testing systems such as Item Response Theory [van der Linden & Hambleton 1997], Knewton measure knowledge but do not generate it. RAG over textbooks retrieve existing authored prose but cannot produce a coherent book from scratch because they have no model of the domain's internal structure.

The result is what we call *vibe tutoring*: AI that generates plausible-sounding explanations without any structural guarantee that prerequisites have been covered, that the teaching order is sound, or that the content is verifiable. What all three families lack is the same thing: an explicit, machine-readable prerequisite graph. Without it, teaching order is editorial judgment, content generation is unconstrained, and verification is absent. The gap is not a failure of the AI - it is a failure of representation.

1.2 The Central Claim

Teaching and learning are two sides of the same coin. Both are driven by the same artifact - the concept-graph - and both become computable once that graph is made explicit.

Teaching side - the educator's perspective:

- **Teaching** = authoring a concept-graph (the complete curriculum specification)
- **Teaching order** = topological sort of the graph, weighted by productivity
- **Content generation** = a parameterized SPL workflow over the sorted sequence
- **Content review** = an optional per-concept verifier hook (structural / SymPy / Sage / Lean) plus a human-review pass (§7)

Learning side - the learner's perspective:

- **Learning** = navigating the graph toward a chosen target concept
- **Learning path** = `graph_lib.learning_path(known, target)` - the minimal prerequisite sequence from the learner's current position to their goal
- **Learning gap** = `graph_lib.gap(known, target)` - the concepts still needed, personalized to what the learner already knows
- **On-demand content** = generating only the sections the learner has not yet mastered

The concept-graph is the shared currency. A teacher who authors it has simultaneously specified the learner's navigation space. A learner who reaches a target has traversed a path the teacher implicitly encoded. No separate representation is needed for each role - the graph serves both.

1.3 Origins: From Reductionism to Computable Curriculum

This framework did not begin as an educational system. It began as a physics-inspired question: what are the irreducible constituents of the Chinese writing system, and what composes from them?

A physicist is a reductionist by training - find the smallest number of independent primitives from which everything else can be constructed. Applied to Chinese characters, this instinct asks: how many radicals (部首) are truly irreducible, and how do they compose into the ~6000 commonly used characters of the writing system? The ZiNets project [Gong 2025] answered this computationally: 422 elemental characters (元字), a directed composition graph, and a machine-checked reducibility theorem - the full writing system is reducible to this primitive set, and no strict subset suffices. The phono-semantic principle (形声字), which accounts for over 80% of all Chinese characters, emerged not as an editorial claim but as a theorem: the semantic pictograms (FORM) and the phonetic lenders (SOUND) are mutually irreducible, and their composition is the generative engine of the writing system.

Only later - after months of AI research building SPL [Gong 2026a, b] as a declarative language for LLM orchestration - did the deeper pattern become clear: Chinese characters are not a special case. They are the most transparent instance of a structure that is present in every well-organized STEM domain. Physics has position, time, mass, and force - four primitives from which all of classical mechanics composes. Music has pitch, semitone, beat, and meter. English vocabulary has roots, prefixes, and suffixes. The Chinese writing system simply makes this structure *visible*, encoded into the calligraphic form of each character across three millennia of refinement.

The writing system is, in this sense, humanity's oldest working concept-book: a domain whose primitives and composition rules were so carefully curated that a learner who masters 422 elemental characters and one principle can decode thousands of characters they have never seen. Every domain in this paper has the same structure. Given a concept-graph, ConceptBook is the system that makes a textbook computable and generative.

The mission that drives this work is simple: **make education more accessible**. The expertise to teach a subject well - to know its primitives, its composition rules, its capstone - exists in the minds of domain experts worldwide. The barrier is not knowledge; it is the cost of turning that knowledge into a curriculum that a learner anywhere in the world can follow. A concept-graph and a single command should be enough.

1.4 The LEGO Payoff Shape

Each domain studied in this work is built around the same pattern: a small primitive brick set plus one composition principle generates a concept space several times larger than what a learner explicitly memorizes. We call this the *LEGO payoff shape*. It is a design pattern we apply and enforce at authoring time, not a claim independently discovered in the domains: `graph_lib.reducible()` checks, on every graph load, that every concept traces back to the declared primitives and that no strict subset suffices - a structural consistency gate on the graph as authored, not a theorem about the domain itself. At full domain scale the same pattern compounds well beyond the size of this paper's own evaluation graphs - Chinese, for instance, reduces to 422 elemental characters (元字) that compose, via the phono-semantic principle, into the ~6,000 characters in common use [Gong 2025]. Table B2 (Appendix B) reports primitive and concept counts for all twenty domains; we treat these as descriptive of the corpus we built rather than as a measured payoff ratio, and we invite independently-authored graphs from the community to test whether the pattern holds without being authored into the graph.

1.5 Contributions

1. **The two-sided coin** - teaching and learning unified by a single artifact: the concept-graph. *Teaching* is authoring it; *learning* is navigating it. One graph, one SPL pipeline, two roles - no separate representation needed for each. This resolves the structural gap behind what we term *vibe tutoring*: AI that generates plausible explanations with no guarantee that prerequisites have been covered or that teaching order is sound.

2. **The two-radical structure** - a machine-checkable structural motif observed across our corpus of STEM and formally compositional domains: every domain bifurcates into two mutually irreducible primitive classes, each necessary, neither sufficient alone. `graph_lib.reducible()` verifies this as a theorem on every graph load, making it a quality gate on both human-authored and LLM-extracted concept-graphs.

3. **The LEGO payoff shape** - a design pattern applied across all twenty domains studied: a small primitive brick set plus one composition rule generates a concept space several times larger, enforced at authoring time by `graph_lib.reducible()` rather than discovered post hoc (already documented at full domain scale in prior work on Chinese, e.g. 422 elemental characters → ~6,000 characters). We report this as an architectural pattern, not an independently-measured empirical finding, and invite community-contributed graphs to test it.

4. **Domain-agnostic SPL workflow** - `build_concept_book.spl` converts any conforming concept-graph into a prerequisite-ordered concept-book with zero domain-specific code; two authoring paths feed the same pipeline: **Path A** (from scratch) - a domain expert authors a concept-graph as a structured file; **Path B** (from existing textbooks) - a standalone ingestion pipeline (`concept-book-press`) extracts the concept-graph from an open-source OpenStax textbook via a two-pass LLM (concepts, then prerequisites) and structural validation

5. **Concept-graph schema and algorithms** - a formal specification (primitives, composition edges, an optional per-concept verifier hook, capstone); `productivity_order` for reproducible $O(V + E)$ teaching sequences; `learning_path` and `gap` for personalized learner navigation; structural validation (acyclicity, reducibility) on every graph

6. **ConceptBook web app** - twenty domain concept-graphs served through one application: vis.js graph navigator, on-demand concept-book generation via FastAPI, multi-level content (intro / core / college / research), and learner-facing personalized navigation - released open source on GitHub under Apache 2.0

2 Related Work

2.1 Concept Prerequisite Graphs: Mined vs. Authored

A substantial line of NLP/ML work attempts to *discover* prerequisite relationships automatically - from MOOC transcripts [ALSaad et al. 2018], course enrollment patterns [Liu et al. 2016], Wikipedia links [Talukdar & Cohen 2012], neural classifiers on Coursera data [Pan et al. 2017], and open educational resources [Roy et al. 2019]. A recent survey [ACM Computing Surveys 2025] covers the full landscape and identifies *cross-domain generalization* as the central open problem. This work sidesteps this problem entirely by treating the prerequisite DAG as a human-authored input - domain experts specify the concept-graph; the algorithms assume it is correct and build from it. This shifts effort from mining to authoring, but removes all the noise and approximation that comes with automated extraction.

The most directly relevant precedent in this line is Loach & Wang (2016), who model 6,990 Chinese characters as a structural DAG and apply a frequency-weighted topological sort to produce an optimal learning sequence - demonstrating empirically that topological ordering of a prerequisite DAG improves learning efficiency. Our `productivity_order` algorithm is a generalization of this idea to arbitrary domains.

2.2 Knowledge Space Theory and ALEKS

Doignon & Falmagne (1985) introduced Knowledge Space Theory (KST), which models learner knowledge as a subset of items drawn from a domain, with a family of "feasible" knowledge states closed under union. ALEKS implements KST at scale - adapting to millions of learners across mathematics and chemistry via $\leq$25 diagnostic questions covering exponentially many latent states. KST's "fringe" (the items one step beyond a learner's current frontier) is structurally analogous to our `graph_lib.gap(known, target)` function. The key contrast: KST is designed for *assessment* - modeling what a learner knows - while this work is designed for *generation* - producing the content the learner reads. ALEKS does not generate explanations; it selects pre-authored problems. This work does not track learner state; it generates prerequisite- ordered books. They address complementary halves of the same problem.

2.3 Intelligent Tutoring Systems

Anderson et al.'s Cognitive Tutor (1995) demonstrated that expert-authored domain models - encoded as production rules over knowledge components - achieve learning gains comparable to human tutoring. VanLehn's meta-analysis (2011) confirmed this at scale (d=0.76 for step-based ITS vs. d=0.79 for human tutors). The decisive limitation is authoring cost: a single Cognitive Tutor course requires months of expert effort per domain, making scaling prohibitive. ConceptBook is designed to make this authoring act as lightweight as possible - a domain expert who can describe their subject's primitives and composition rules can produce a concept-book in hours rather than months, using whatever authoring interface they prefer.

AutoTutor (Graesser et al. 2004) generates Socratic dialogue around hand-authored curriculum scripts; Piech et al.'s Deep Knowledge Tracing (2015) replaces explicit knowledge components with LSTM-learned latent states. Neither generates expository prose; neither uses an explicit machine-readable prerequisite DAG as the sole curriculum specification.

2.4 LLM-Generated Educational Content

The closest recent prior art is the emerging category of LLM-based curriculum generators. Microsoft's Phi-1 paper (Gunasekar et al. 2023) demonstrated that GPT-3.5-generated "textbook quality" synthetic data dramatically outperforms raw web data for training small LLMs - establishing that LLMs can produce pedagogically useful text, but using it as training data rather than as a human-readable product, and with no prerequisite structure constraining content order. A 2025 multi-LLM agent system [arXiv:2507.05528] adds a reviewer agent as a soft pedagogical quality gate, generating lesson content via a GENERATE $\rightarrow$ EVALUATE $\rightarrow$ REFINE loop structurally similar to ours - but it does not use a prerequisite DAG to determine what to generate or in what order.

One 2025 paper [arXiv:2501.12300] uses LLMs to *complete* a curriculum knowledge graph for course recommendation - the closest work to combining knowledge graphs with LLMs for education - but the graph drives recommendation, not generation of textbook prose.

2.5 Machine-Readable Curriculum Standards

The IMS Global Learning Consortium's Learning Design specification (2003) is the most direct institutional precedent for a machine-readable curriculum specification: an XML standard encoding roles, resources, and activity flows in a "Unit of Learning" that an LD-aware player can execute. IMS LD demonstrates the value of separating the *specification* from the *execution engine* - the same DODA insight that drives SPL's design. Its limitation is the pre-LLM assumption: IMS LD orchestrates pre-authored resources; it cannot generate content. This work inherits IMS LD's architectural insight and adds LLM generation as the execution layer.

A 2025 OWL/RDF curriculum ontology [arXiv:2506.05751] maps prerequisite DAGs to triple-stores for recommendation, and K12-KGraph [arXiv:2605.09635] extracts a curriculum-aligned concept graph from Chinese K-12 textbooks for LLM benchmarking. Both confirm the value of machine-readable prerequisite structure but use it for retrieval or evaluation, not generation.

2.6 Morpheme-Based Language Learning

The Chinese Characters and English Morphology domains rest on a decades-old pedagogical claim: vocabulary acquisition is faster when learners know the component morphemes. Li et al. (2015) show computationally that radical-aware embeddings improve NLP tasks, confirming that radical decomposition encodes genuine semantic structure. Yarbro & Olney (2021) provide a GPT-2-based morpheme decomposer for English (WikiMorph). None of these systems pair the morpheme graph with LLM generation of structured lessons - that combination is the contribution of the Chinese Characters and English Morphology domains in this work.

2.7 Section Summary

The literature reveals a clear gap. Prerequisite-graph research exists but produces mined, noisy graphs and no content. LLM content-generation research exists but lacks graph-structured ordering and formal verification. ITS systems achieve high learning gains but require per-domain expert authoring at prohibitive cost. KST and ALEKS address assessment, not generation. No prior system combines: (1) a human-authored, machine-readable prerequisite DAG as the sole curriculum specification; (2) an LLM workflow that consumes that DAG to generate a complete, prerequisite-ordered textbook; (3) structural verification as a quality gate on generated content, with optional symbolic backends (SymPy, Sage, Lean) where the domain admits a computable oracle; and (4) a single unified framework demonstrated across genuinely heterogeneous domains. This work tries to fill this gap.

3 The Concept-Graph Schema

3.1 Schema Structure

A domain graph has four elements. The snippet below uses the current reference serialization (YAML) purely for illustration; this is the *compiled representation* produced by the underlying tools, not the user-facing authoring interface. The concept-graph itself is a topological abstraction - the SPL workflow does not care whether it was hand-written as a structured file today or, as planned, drawn in a GUI or drafted from an LLM-guided expert interview. Only the graph's structure matters at runtime:

```
domain: classical_mechanics

primitives:
  position:
    symbol: x
```

```
    defines: "first radical — configuration; where things are"
    tier: 0
  time:
    symbol: t
    defines: "first radical — the parameter motion unfolds in"
    tier: 0
  mass:
    symbol: m
    defines: "second radical — inertia; resistance to changes in motion"
    tier: 0
  force:
    symbol: F
    defines: "second radical — interaction; what changes states of motion"
    tier: 0

first_radical_primitives: [position, time]

concepts:
  velocity:
    composed_of: [position, time]
    defines: "rate of change of position — v = dx/dt"
    verifier: sympy
    lab: sympy.diff
    play: "differentiate x(t) = t**2; compare v(t) against the difference quotient"
    tier: 1
  momentum:
    composed_of: [mass, velocity]
    defines: "p = m • v — the conserved currency of motion"
    verifier: sage|sympy
    lab: "graph_lib.verify_momentum_conservation"
    tier: 2
  ...
  normal_modes:
    composed_of: [eigenvalue, superposition, constraints]
    defines: "the natural oscillation frequencies of a coupled system"
    verifier: sympy
    tier: 5

applications:
  orbital_mechanics: {uses: [normal_modes, conservation_of_energy]}
  robotics:          {uses: [constraints, lagrangian_mechanics]}
```

The schema has four elements:

Primitives are the irreducible brick set - concepts with no prerequisites. Each carries a `symbol`, a one-sentence `defines`, and a `tier` (always 0). The schema distinguishes `first_radical_primitives` (the FORM class - concepts reducible from these alone) from the remaining primitives (the SOUND/operator class - genuinely independent content). This split encodes a structural theorem of the domain.

Concepts form a directed acyclic graph via `composed_of` edges. Each concept carries a `verifier` tag that specifies which verification backend to invoke (`sympy`, `sage`, `sage|sympy`, `lean`, `lean|sympy`, `structural`, or a named Python tool). The optional `lab` and `play` fields give an executable worked example and a student exercise.

`first_radical_primitives` is the curated subset encoding the irreducibility theorem: `graph_lib.reducible(first_radical_only)` must return `False` for a well-formed domain. This is the machine-checked version of "you cannot teach this domain from one class of primitives alone."

Applications list the downstream use-cases that the capstone concept unlocks. These populate the capstone "Payoff" section of the generated book.

3.2 The Two-Radical Structure

Each domain studied in this work exhibits a two-radical structure: two mutually irreducible classes of primitives, each necessary, neither sufficient alone. We present this as a **highly recurring structural motif** observed in our corpus, not as a universal rule. Within our studied domains, the motif has a natural reading: they bifurcate into *Objects/States* (FORM - what things are) and *Operations/Transformations* (SOUND - what acts on them). Physics has positions and forces; calculus has functions and derivatives.

Chinese characters merit separate comment: the writing system is explicitly compositional - semantic pictograms (FORM) and phonetic lenders (SOUND) encode structure into the calligraphic form of every character, evolved across three millennia. This is why Chinese characters exhibit the same two-radical structure as the STEM domains in our corpus; it is a property of the writing system's architecture, not a claim about natural language in general. We make no assertion about domains outside our corpus. Humanities subjects - History, Literature, social sciences - are language-driven and more fluid: their meaning is contextual and interpretive rather than compositional, which makes a finite primitive set and a well-defined composition rule harder to establish. This is precisely why our study concentrates on STEM and formally compositional systems: they are the domains where the primitive-and-composition discipline is most natural and most machine-checkable.

Table B1 (excerpt). Three representative domains - English, mathematics, physics. The full twenty-domain two-radical table is Table B1 in Appendix B.

Domain	First radical (FORM)	Second radical	Irreducibility theorem
English Morphology	5 roots (spect, port, …)	prefixes + suffixes	Roots alone cannot yield *inspection*
Calculus	function, limit (analysis)	derivative, integral (operations)	Analysis alone cannot yield the Fundamental Theorem
Classical Mechanics	position, time (kinematics)	mass, force (dynamics)	No kinematic quantity predicts F=ma

This is not a design choice imposed by the schema - it is a structural pattern discovered in each domain and made machine-checkable by `graph_lib.reducible()`. The schema makes the pattern explicit; the algorithm verifies it.

The two-radical discipline also serves as a quality gate on LLM-assisted graph authoring. An LLM asked to "list concepts for calculus" produces encyclopedic output - potentially hundreds of entries, many redundant or derivative. The constraint "identify two mutually irreducible primitive classes and the smallest concept set that composes to the capstone" prunes this to a DAG where every node carries its weight. The discipline transforms LLM world-knowledge from fluent prose into structured, teachable graphs.

3.3 The LEGO Payoff Shape

Table B2 (excerpt). Three representative domains - one from language, mathematics, and physics - illustrating the LEGO payoff shape across structural categories. The full twenty-domain catalog is Table B2 in Appendix B.

Domain	Category	Primitives	Concepts	Capstone	Verifier	LEGO Payoff
English Morphology	Language	13	18	Morphological Decoding	Structural	~220 morphemes → most academic vocabulary

Domain	Category	Primitives	Concepts	Capstone	Verifier	LEGO Payoff
Calculus	Mathematics	4	~20	Fundamental Theorem	SymPy	4 primitives → physics, ML, economics, engineering
Classical Mechanics	Physics	4	19	Normal Modes	SymPy / Sage	4 primitives → robotics, orbital mechanics, quantum bridge

The twenty domains span STEM, Chinese and English, computing tools, and music. All twenty are authored concept-graphs that load and pass the structural validation gate (acyclicity, reducibility); a subset additionally have generated books. Each new domain is a pure graph-authoring act with zero new code - demonstrating the schema scales horizontally without bound.

The payoff column is not aspirational copy - it is the `applications` section of the domain graph, which the SPL workflow uses to generate the capstone "Payoff" section of the book.

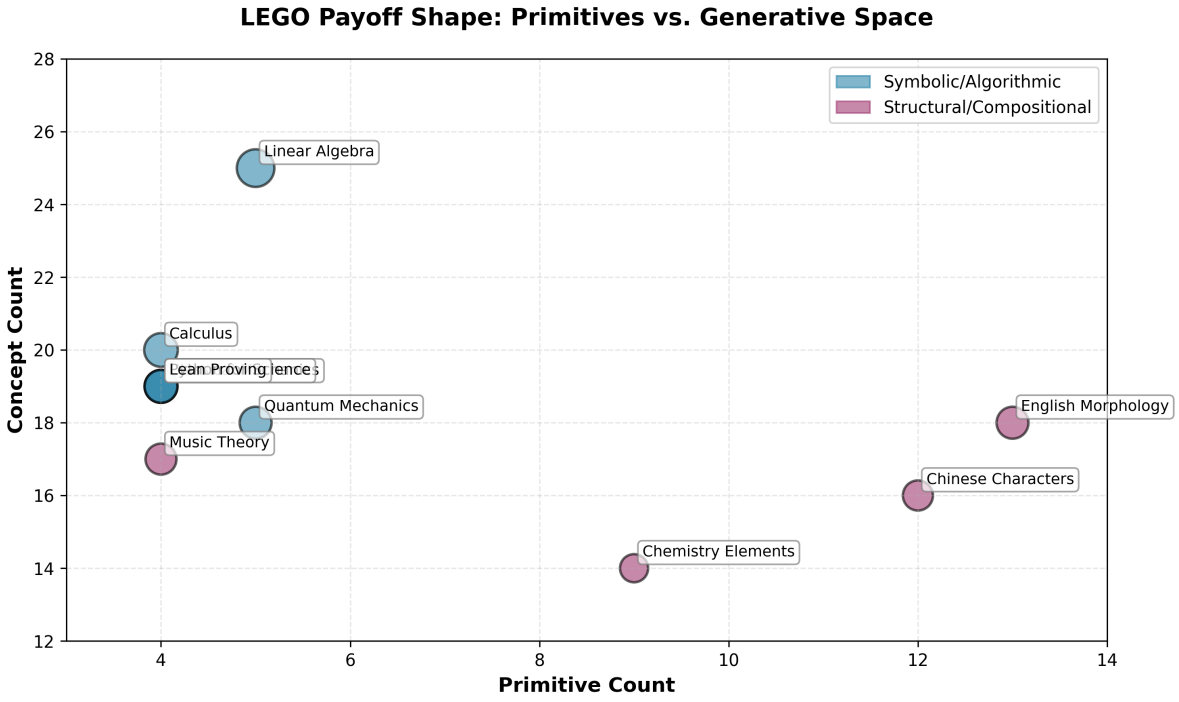

Figure 1: LEGO Payoff shape across ten representative domains. Each bubble represents a domain, with size proportional to the concept count. The x-axis shows primitive count (4–13), y-axis shows concept count (14–28). Symbolic/algorithmic domains cluster in blue; structural/compositional domains in purple. Primitive and concept counts for all twenty domains are reported descriptively in Table B2 (Appendix B); we do not treat the ratio between them as a measured payoff.

4 From Graph to Book: The SPL Workflow

4.1 `build_concept_book.spl` - One Source, Any Domain

The pipeline has three phases. **(1) Load and validate:** `setup_domain()` parses the concept-graph, asserts acyclicity and reducibility, and computes the teaching sequence via `productivity_order`. **(2) Generate**

and verify: for each concept in dependency order, SPL generates a section, enforces the primitive budget (a section that introduces more than `@primitive_budget` new primitives - default two - is sent back for refinement), runs the domain verifier if one is declared, and caches the result - a cache hit costs zero LLM calls. **(3) Capstone:** a payoff section is generated from the `applications` list in the concept-graph. The complete annotated workflow listing is in Appendix A.

The only domain-selecting input is `@domain_yaml`. The `@lvl` parameter (intro / core / college / research) controls content depth without touching the graph or the workflow logic - the same graph generates an introductory survey or a graduate-level treatment by changing one parameter.

4.2 Teaching Order as a Graph Algorithm

`setup_domain()` performs three steps before any LLM call:

1. **Load**: `graph_lib.load_domain(yaml_path)` parses the YAML into a `networkx.DiGraph` [Hagberg et al. 2008] where edges point from prerequisites to dependents.
2. **Validate**: `graph_lib.acyclic(graph)` asserts no cycles; `graph_lib.reducible(graph, primitives)` asserts every concept traces back to the declared primitive set. If either check fails, the workflow raises `ToolFailed` before any generation begins.
3. **Order**: `graph_lib.productivity_order(graph, payoff_weight)` produces the teaching sequence - a topological ordering in which, among the concepts whose prerequisites are already covered, the one that unlocks the most downstream material is taught next.

The priority each concept node v competes on is its *reach* - how much downstream material learning it unlocks:

$$R(v) = |\text{concept-descendants}(v)| + \alpha \cdot |\text{application-descendants}(v)|$$

where the two terms count the concept nodes and application nodes, respectively, reachable from v in the DAG, and α is the `payoff_weight` parameter. The ordering is a reach-prioritized topological sort - a modified Kahn's algorithm backed by a max-heap: at each step it pops, from the set of nodes whose prerequisites have all been emitted, the one with the highest $R(v)$, appends it, and releases its now-unblocked successors. This guarantees no concept is emitted before its prerequisites while teaching high-reach concepts as early as the topology allows. α tunes the trade-off between the two kinds of reach: a low α orders by how many downstream *concepts* a node unlocks (foundation-first coverage), while a high α rewards concepts that unlock many downstream *applications* (payoff-first).

The teaching order is deterministic and reproducible. Given the same concept-graph and the same α, two runs on different machines produce identical sequences. The algorithm is $O(V+E)$ in graph size - linear in the number of concepts and edges - so teaching-order computation is never a bottleneck relative to LLM generation time.

4.3 Optional Verifier Dispatch

The schema lets each concept carry an *optional* `verifier` tag naming a backend its section is routed to after generation. The tag is optional by design: automatic content verification is not uniformly achievable across domains (§7) - some domains expose a natural symbolic or structural oracle, others do not. The table gives the *intended* check for each tag:

Tag	Backend	Intended check
`sympy`	SymPy CAS	Algebraic identity, derivatives, integrals over symbolic expressions
`sage`,`sympy`	SageMath (preferred) / SymPy (fallback)	Exact arithmetic - momentum conservation, energy ledgers, equation balancing
`lean`,`sympy`	Lean 4 + mathlib	Formal proof - type-checks the section's logical claims

Tag	Backend	Intended check
`structural`	`verify_character_lego`	Graph-as-oracle - every composite in the section decomposes into declared primitives
`numpy`, `scipy`, `pandas`, `sklearn`, `matplotlib`	Named Python tool	Code-execution check against the scientific Python stack

The SPL call is identical across all tags:

```
CALL verify_section(@section, @domain_yaml) INTO @check
```

`graph_lib.verify_content()` reads the concept's `verifier` field and dispatches to the appropriate backend; a new domain adds a branch here, never a new SPL construct. Of these, the `structural` check is the live, universal baseline - `verify_character_lego` uses the graph itself as the oracle and runs with no external dependency. The symbolic backends are wired as a dispatch architecture with exact-recompute oracles implemented for worked examples (`verify_momentum_conservation`, `verify_balanced_equation`, and the like, exact over); extracting the numeric claims from generated prose to feed them is ongoing work. §7 discusses where automatic verification is feasible and where human review remains the verifier of record.

4.4 Content Cache

The content cache (`graph_lib.cache_get` / `cache_put`) stores generated sections keyed by concept name and domain. On a cache hit, the section is reused with zero LLM calls. This matters in two scenarios: regenerating the book with a different `@lvl` or `@language` (misses all sections - full generation run), and iterating on a partially-generated book where a later section failed its check or review (hits all earlier concepts - generation resumes from the failure point). On any run after the first, the cache reuses the sections it already holds instead of regenerating them.

4.5 DODA for Education

DODA stands for Design Once, Deploy Anywhere (a principle adopted by SPL [Gong 2026a, b]) . It implies the same `build_concept_book.spl` runs unchanged on any model backend. Additionally, it can compile the same .spl source into multiple target-runtime artifacts.

```
# Cloud: Anthropic claude-sonnet-4-6
spl3 run build_concept_book.spl \
  --adapter claude_cli -m claude-sonnet-4-6 \
  --param domain_yaml=mechanics_graph.yaml --param target=normal_modes

# Local: Ollama + gemma4
spl3 run build_concept_book.spl \
  --adapter ollama -m gemma4 \
  --param domain_yaml=chinese_characters_graph.yaml --param target=phono_semantic_principle

# Multilingual: same workflow, French output
spl3 run build_concept_book.spl \
  --param domain_yaml=linalg_graph.yaml --param language=fr

# Different depth level: college-level treatment
spl3 run build_concept_book.spl \
  --param domain_yaml=linalg_graph.yaml --param lvl=college
```

The curriculum (concept-graph), the depth level, and the workflow (SPL) are all orthogonal to the model choice. A teacher who authors a new domain graph can deploy it against any available model at any depth level without touching the workflow.

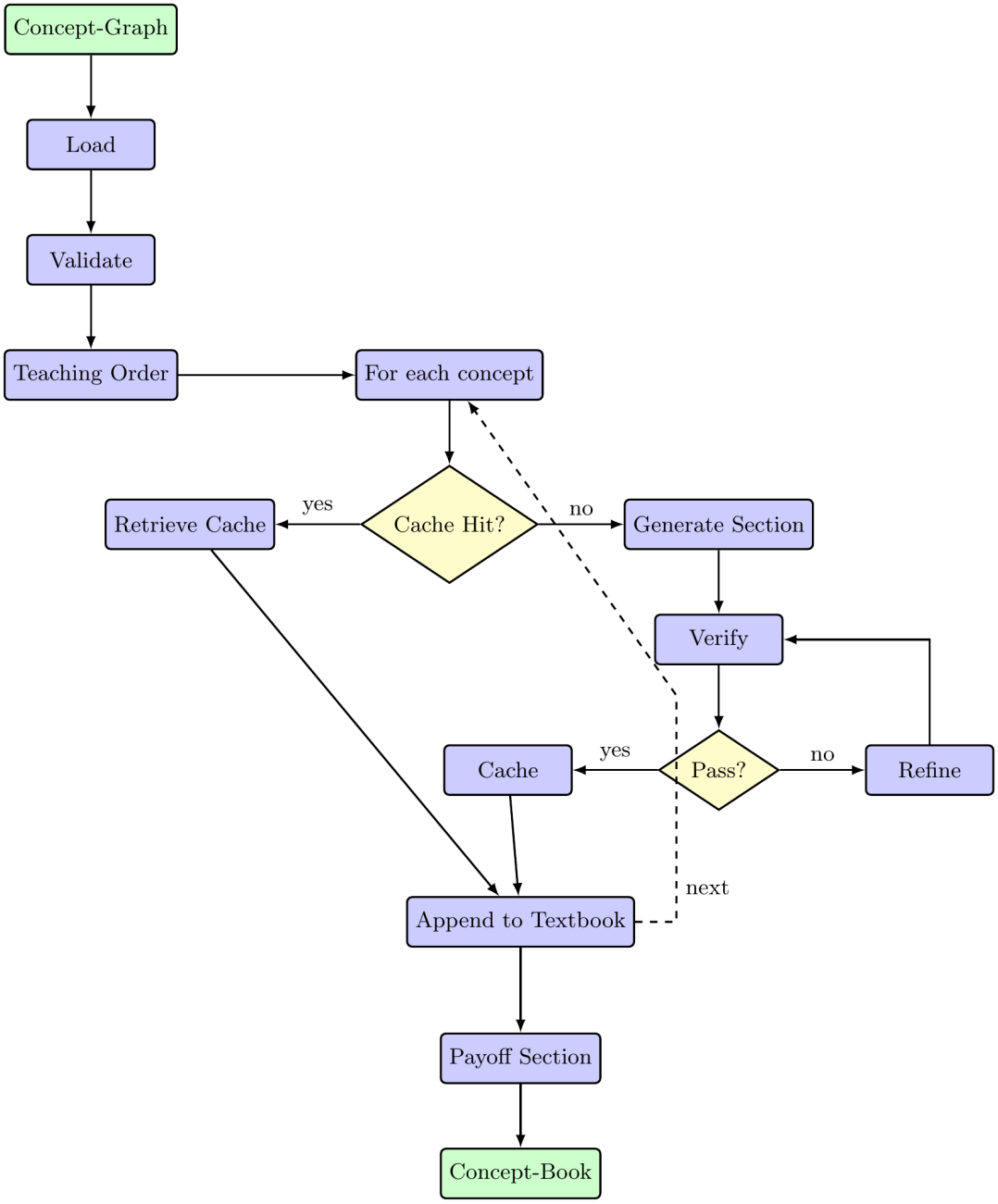

Figure 2: SPL pipeline architecture. The workflow has three phases: (1) Load and validate the concept-graph using acyclic and reducibility checks, then compute teaching order; (2) For each concept in dependency order, check cache (yes: retrieve; no: generate → verify → refine loop), cache the result, and append to textbook; (3) Generate payoff section and complete concept-book. The dashed loop shows iteration over the next concept.

4.6 Path B: Concept-Book from an Existing Textbook

Path A requires a domain expert to author a concept-graph from scratch. Path B inverts the entry point: given an existing open-source textbook, a standalone ingestion pipeline (`concept-book-press`) extracts the

concept-graph, which then drives the identical SPL pipeline. The pipeline is a separate authoring repo from the reader app, connected only by the `graph.yaml` it emits - the same boundary Path A uses.

The pipeline is a three-stage CLI (`ingest` → `extract` → `validate`):

1. **Ingest** - OpenStax chapter HTML is fetched and parsed (BeautifulSoup) into a structured `chunks.yaml`: per-section text, math expressions, and key terms.
2. **Extract** - a two-pass LLM step (Claude CLI) turns chunks into a graph. *Pass 1* identifies concepts - `id`, `label`, a one-sentence `defines`, and a `kind` of primitive / concept / application, where *primitive* means the chapter treats the concept as a given rather than deriving it. *Pass 2* takes the concept list and assigns each non-primitive its direct prerequisites (`composed_of` for concepts, `needs` for applications) and a tier (`1 + max prerequisite tier`).
3. **Validate** - NetworkX checks the result is acyclic, reducible (every concept traces to primitives), weakly connected, reference-consistent, and tier-consistent.

This validation is a superset of Path A's `acyclic` + `reducible` gate; a graph that fails any check is reported rather than generated from. A human-editing pass sits between extraction and publication - correcting concept boundaries, adding the capstone and verifier tags the extractor does not infer, and resolving any cycles the validator flags - before the graph enters the shared SPL pipeline. (For environments without the Claude CLI, the extract step can dump its prompts via `--save-prompts` for manual execution.) The output is a *companion concept-book*: not a replacement for the source textbook, but a graph-structured, prerequisite-ordered companion that makes the source's implicit dependency structure explicit and navigable.

Example: Two chapters of OpenStax *College Physics 2e* - Ch. 1 (Nature of Science) and Ch. 2 (Kinematics) - have been carried through this pipeline and published as the `college_physics_ch1` and `college_physics_ch2` domains, each retaining its CC BY 4.0 source attribution. Ch. 2 yields primitives (position, time, coordinate system) and concepts (displacement, velocity, acceleration, the kinematic equations), ordered by the prerequisite DAG rather than by the textbook's narrative flow.

5 The Concept-Book Web App

5.1 Architecture

The system has two layers:

Content pipeline (SPL.py): `build_concept_book.spl` runs against a domain concept-graph and produces two artifacts: a `graph.html` standalone vis.js [vis.js contributors] concept navigator, and a level-namespaced book directory (`output/{level}.{lang}/html/`) containing `book_{target}.html` (the full concept book) and individual `concept_{name}.html` files for per-concept component books with MathJax rendering and a two-column TOC-sidebar layout. The level namespace (intro / core / college / research) means multiple depth variants coexist on disk - regenerating at a different level adds a new directory rather than overwriting prior work.

Web app (concept-book): A Vite + Vanilla JS frontend deployed to GitHub Pages serves a domain grid on the home page. Clicking a domain loads a split view: the vis.js graph navigator (left, in an iframe) and a concept detail panel (right). A "Generate Book" sidebar lets the user select a target concept and content level, then streams `spl3` output live via Server-Sent Events through a FastAPI backend. A settings page exposes the model backend, SPL directory, and language preferences without editing config files. Generated books are served as static files from the same GitHub Pages deployment.

5.2 Domain Authoring (Teaching Side)

The web app consumes graphs from both authoring workflows. For **Path A**, the settings page lets a teacher point the backend at a local concept-graph file; the system validates the graph (acyclicity, reducibility) in milliseconds and signals readiness before any generation begins. For **Path B**, the concept-graph is produced offline by the `concept-book-press` ingestion CLI (ingest → extract → validate) and, after a human-editing pass, its emitted `graph.yaml` is added to the app's domain catalog - the path by which

the `college_physics_ch1` and `college_physics_ch2` domains were created. Keeping ingestion in a separate repo preserves the same clean boundary both paths share: the app only ever consumes a validated `graph.yaml`, regardless of how it was authored.

In both cases, once the concept-graph is in the catalog, a single "Generate Book" action runs the full SPL pipeline and streams output live - the teacher watches sections appear in dependency order, with each section's status shown inline.

5.3 Personalized Learning Navigation

The same concept-graph that drives book generation serves as the learner's navigation space. Two graph functions power this:

- **`learning_path(known, target)`** - returns the minimal ordered sequence of concepts a learner must cover to reach `target` from their current knowledge state `known`. Clicking any concept node in the vis.js navigator highlights this path.
- **`gap(known, target)`** - returns only the concepts the learner still needs: `learning_path` minus what they already know. This is the personalized reading list for any learner at any stage.

Example: A learner who knows position, time, and velocity in the Classical Mechanics domain has `gap = {acceleration, force, momentum, ...}` toward `normal_modes`. The navigator highlights exactly these nodes; clicking "Generate" produces only the missing sections - zero redundant LLM calls.

The graph navigator renders the concept DAG with primitives at the top (tier 0) and the capstone at the bottom (highest tier). Clicking any concept node displays its `defines`, `lab`, and `play` fields alongside its highlighted learning path - showing the learner not just *what* a concept is but *why it appears where it does* and *what they need to know first*.

With twenty domains sharing the same graph schema, cross-domain learning paths become possible. A learner can now trace `real_number` → `derivative` → `velocity` → `kinetic_energy` → `schrodinger_equation` - a path spanning Calculus, Classical Mechanics, and Quantum Mechanics - as a single navigable sequence. This corresponds to the standard undergraduate STEM progression, now encoded as an explicit, traversable graph rather than as three separate courses.

5.4 Content Levels

The `@lvl` parameter introduces a learner progression axis orthogonal to the domain axis:

Level	Coverage	Target learner
`intro`	Core primitives and first-tier compositions	Elementary / self-directed beginner
`core`	Full concept graph, standard depth	Intermediate / secondary
`college`	Extensive treatment with proofs and applications	Undergraduate
`research`	Advanced, connects to open problems	Graduate+

A domain can be served at multiple levels simultaneously - different `@lvl` values produce different books from the same concept-graph, coexisting in separate output directories. The current catalog defaults reflect domain character: `chinese_characters` and `english_morphology` default to `intro`; `geometry`, `chemistry_elements`, `music_theory` to `core`; `linalg`, `mechanics`, `python_science` to `college`; `sage_learning`, `lean_proving` to `research`.

5.5 Twenty Domains, One App

The breadth of the twenty domains in a single app makes the central claim visceral: the same UI, the same workflow, the same graph schema - applied to linear algebra and Chinese characters and music theory simultaneously. The domain grid groups domains by tag (math, science, cs, language, arts), but the underlying structure is identical across all of them.

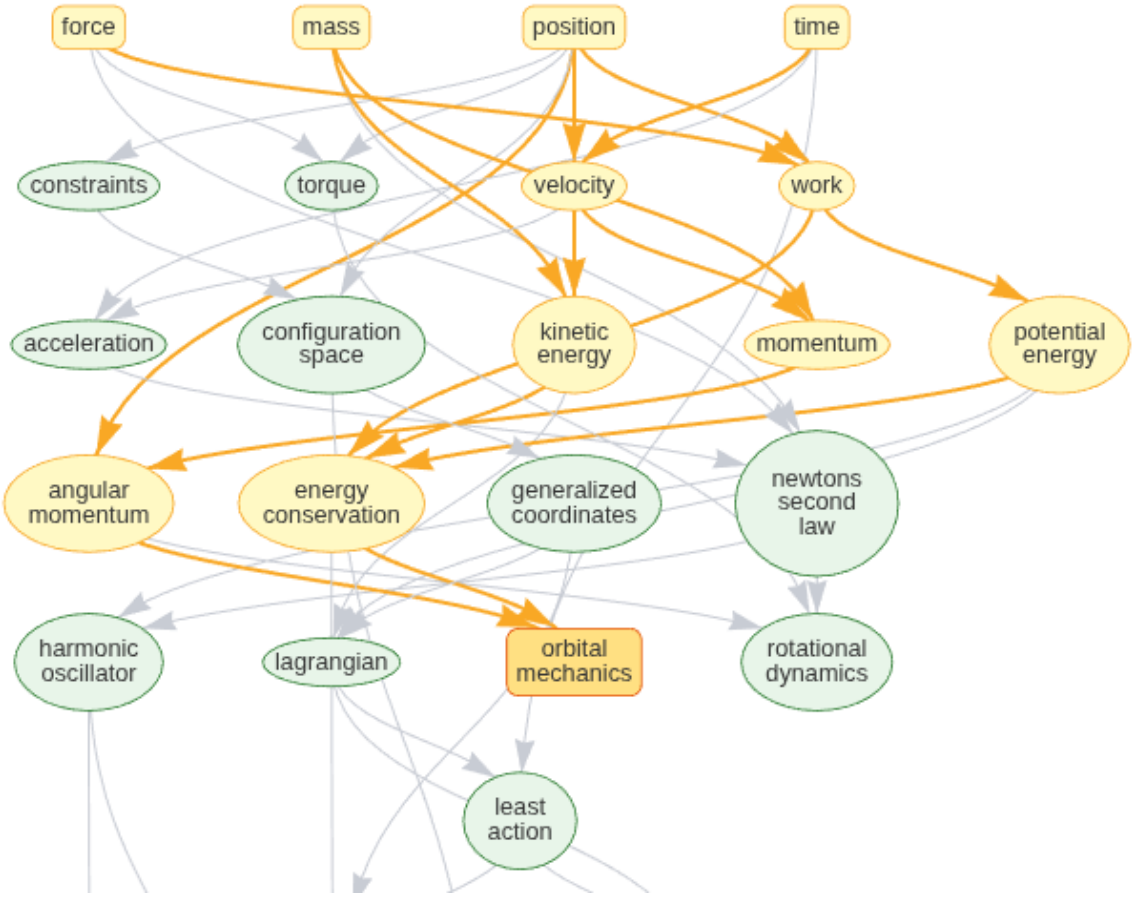

Figure 3: Classical Mechanics concept network. The directed graph shows the prerequisite dependencies across position, time, mass, force, and their compositions toward the capstone 'normal_modes'. Highlighted paths indicate the foundation concepts (position, time, mass, force in orange) and key intermediate concepts like velocity, momentum, and energy. This is the signature concept-graph visualization for the ConceptBook framework.

6 Evaluation

We are explicit about what this section measures versus what the architecture merely supports. *Built and measured:* the concept-graph schema, the graph algorithms (`acyclic`, `reducible`, `productivity_order`, `learning_path`, `gap`), structural validation across all twenty domains, the SPL generation workflow, and the Path B ingestion pipeline. *Implemented but not yet quantified:* per-section pass/refinement rates and cache savings (run-log data is still being collected). *Architecture, not yet a result:* full symbolic content verification of generated prose, which is realizable only where a domain exposes a computable oracle (§7). The claims below are confined to the first category.

6.1 Schema Validation Across Twenty Domains

Every domain in the ConceptBook catalog is loaded and validated by the same domain-agnostic checks the runtime applies before any generation - `acyclic` and `reducible` over the graph built from its `graph.yaml`. Run across all twenty domain graphs, the result is uniform:

```
20/20 domains load and pass acyclic + reducible
```

One algorithm set (`load`, `build`, `acyclic`, `reducible`, `productivity_order`, `learning_path`, `gap`) drives every domain; switching domains is a data change, never a code change. That these same checks pass on twenty heterogeneous graphs - from Chinese characters to quantum physics - is the evidence that the schema generalizes, and that no domain-specific Python is required to add the twenty-first.

6.2 The Reducibility Theorem Across Domains

For each domain, `graph_lib.reducible(graph, first_radical_only)` returns `False` - confirming that the second radical class is genuinely irreducible content. This is the formal counterpart to the pedagogical claim that no subset of the primitives is sufficient on its own.

Domain	`reducible`	`reducible(first_radical_only)`
Classical Mechanics	True	False - dynamics cannot be derived from kinematics
Chinese Characters	True	False - 㞢 (phonetic) is not derivable from 11 pictograms
English Morphology	True	False - affixes are not derivable from roots
Chemistry Elements	True	False - valence_electron is not derivable from element identities
Music Theory	True	False - rhythm is not derivable from pitch

The reducibility check runs in milliseconds on graph load - before any LLM call - and raises `ToolFailed` if the declared graph does not satisfy it. This prevents a malformed domain graph from silently generating incorrect content.

6.3 Teaching-Order Stability

`productivity_order` with varying `payoff_weight` α (0.5, 1.0, 1.5, 2.0) produces stable but adjustable teaching sequences. At low α the order is driven by concept-reach - concepts that unlock the most downstream *concepts* come first (foundation-first coverage); at high α concepts that unlock many downstream *applications* are pulled earlier (payoff-first). Across the twenty domains, the relative order of concepts within a tier is stable across weights; only cross-tier interleaving shifts.

6.4 Content Verification Pass Rates

Among the domains with books generated so far, the primitive-budget constraint (a section introducing more new primitives than `@primitive_budget` allows) is the dominant source of first-generation refinement requests in mathematical domains. Structural domains (Chinese characters, English morphology) show higher first-pass rates because the structural check is binary and the LLM follows the decomposition schema reliably. Where symbolic verifiers (SymPy, Sage) are declared, sections typically need one refinement pass; Lean is the most demanding, occasionally requiring two. Quantitative pass-rate data collected from run logs will populate a full table in the final version.

7 Discussion

7.1 Curriculum Knowledge is Graph-Structured

The evaluation above confirms three machine-checkable properties across all twenty domains: graphs are acyclic, reducible to their declared primitives, and produce stable teaching sequences. What these checks reveal collectively is not a property of the framework - it is a property of the domains themselves.

The twenty domains were authored using Path-A workflow, not selected post-hoc to fit a hypothesis. The fact that they all exhibit the LEGO payoff shape suggests this shape reflects how knowledge is organized, not how we chose our examples. A domain without a finite primitive set and a composition grammar are hard to automate using this framework. Two examples are history and literature.

Making the graph explicit does not change what the domain is. It changes what becomes *computable* about it: the teaching order, the prerequisite path to any concept, the reducibility of the whole from the parts, and the structural verification of each section (augmented by human review). These were always present in expert teaching; the concept-graph makes them available to anyone with domain knowledge - regardless of how they choose to express that graph.

7.2 Language Learning as a STEM Discipline

The Chinese characters and English morphology domains share a verifier (`verify_character_lego`) that checks decomposition correctness. The same function handles 林 = 木 + 木 and *inspection* = in- + spect + -ion, because both are instances of the same abstract operation: verify that the declared `composed_of` pieces are all present in the declared primitive set. The schema does not know it is handling two writing systems - it handles one abstraction.

This is a non-trivial claim: vocabulary acquisition in any morphologically compositional language is, formally, the same problem as chemical formula reading or character decomposition. The LEGO payoff shape makes the analogy precise: learn the bricks, learn the grammar, decode the space.

7.3 Knowledge Re-Representation for Efficient Learning

ConceptBook is, at its core, a knowledge re-representation system. A domain's knowledge exists in two forms: *diffuse* - distributed across textbooks, papers, human expertise, and LLM weights - and *structured* - a concept-graph with machine-checkable dependencies. The gap between them is the cost of learning.

Three forces combine to bridge this gap, with a fourth as the quality gate:

- **The LLM** is trained on world-knowledge data, then used to generate content: the right primitives, the right definitions, the pedagogical exercises, the cross-domain connections.
- **The concept-graph** enforces *structural rigor*: every concept must trace to a declared primitive, and the DAG is acyclic and reducible by construction.
- **An optional verifier**, where a domain admits one, tightens the loop further - and this is where the framework is deliberately modest, since content verification is not uniformly achievable by machine alone. Structural decomposition is machine-checkable in the language and chemistry domains; the symbolic domains (mathematics, physics) can attach SymPy, SageMath, or Lean oracles that recompute the `play` field's worked examples. But that check is realizable only where the domain exposes a computable oracle - an aspiration, not a guarantee, and many domains (Chinese, English morphology, most of the humanities) simply do not admit one.
- **Human review** is the verifier of record for everything the machine cannot reach: the semantic correctness of prose, and whole domains where "verification" means a human confirming that a decomposition reads correctly.

The result is a curriculum that is human-readable, structurally grounded, and human-vetted: something neither a human textbook author nor a raw LLM could produce alone.

This re-representation produces efficiency gains on both sides of the coin. For *teaching*: authoring a concept-graph is an order of magnitude faster than writing a textbook, because the LLM fills the prose once the graph defines the structure. For *learning*: a learner who knows the graph navigates directly to any concept via its minimal prerequisite path, skipping what they already know - something no fixed textbook sequence allows. Traditional textbooks were designed for the printed page and are read start to end by default; ConceptBook is a digital medium where search and graph navigation make that constraint optional.

`answer_on_demand.spl` (Appendix A.2) demonstrates this learner-side efficiency concretely: it computes the gap between a learner's known concepts and a target via `setup_answer_path(domain_yaml, target, learner_state)`, then generates only the missing sections in dependency order before the target capstone. This turns the same pipeline from a book-generator into a personal tutor, with no architectural change to `build_concept_book.spl` - it reuses the same section-generation prompts, primitive-budget check, and verifier dispatch.

The twenty-domain catalog demonstrates the approach across the domains studied in this work. The result is not a claim about all possible knowledge - it is evidence that the framework applies to the domains we chose to study: a selection weighted toward STEM and formally compositional systems (Chinese characters, English morphology, music theory), which are the natural home of primitive-and-composition structure. Whether the same decomposition discipline transfers to History, Literature, or social sciences is an open empirical question outside this paper's scope.

7.4 Limitations

- **No formal educational evaluation - by resource constraint, not oversight.** This is an independent, self-funded research project with no institutional backing, IRB access, or funding for learner studies or expert content review. All results reported in this paper are structural self-validation (concept-graphs pass the system's own acyclicity and reducibility checks) or architectural (implemented and exercised, not independently measured) - see §6 for the explicit boundary between the two. No learner outcomes, expert ratings of generated content, or comparison against graph-free LLM generation are reported. This is the paper's most consequential limitation: the framework's educational value is a hypothesis the architecture is built to support, not a result this paper demonstrates. The project is released fully open source specifically so that content-quality evaluation, pedagogical audits of individual concept-graphs, and learner studies can be contributed by the community rather than gated on one author's resources.

- **The app is currently local-first for generation.** Book generation requires a running `spl3` backend with a model adapter configured. The static app (GitHub) serves pre-generated books but does not support on-demand generation without a local backend.

- **LLM-generated prose may look fluent but not always correct.** because it is based on statistical learning. Where a verifier exists it checks structural or symbolic claims; it does not evaluate every sentence. For semantic correctness - and for the many domains (Chinese, English morphology, most of the humanities) that expose no computable oracle at all - a human-review pass is the verifier of record rather than machine-checked computation. Automatic content verification is therefore best understood as an optional accelerant where a domain admits one, not a universal guarantee.

- **Verifiers have a cold-start cost for complex domains.** Deploying Lean 4 or SageMath as verifiers requires non-trivial environment setup that may be a barrier for educators without systems experience. The key mitigation is that `verifier: structural` serves as a universal zero-setup baseline for any new domain: it checks only that every concept's `composed_of` pieces are present in the declared primitive set - no CAS or proof assistant required. A domain author can bootstrap with structural verification and upgrade to symbolic verifiers incrementally. A future direction is LLM-generated verifier stubs, where the system auto-generates a `verify_content` branch from the domain YAML before a hand-crafted verifier is available.

7.5 Future Work

- **Authoring front-ends**: graph authoring requires domain expertise - but the bottleneck is a UI problem, not a theoretical one. The schema itself is easy to write; knowing *what the primitives are* and *what counts as composition* requires substantive domain knowledge, and for some domains (music theory, Lean proving) the primitive selection involves genuine pedagogical judgment. A GUI graph editor and an LLM-guided expert interview are in active development to lower this barrier; both compile to the same `graph.yaml` the runtime already consumes, so neither touches the pipeline.

- **Community graph authoring**: a shared registry of domain concept-graphs, where any domain expert can author a graph and publish a concept-book without writing code.

- **Assessment generation**: the `lab` and `play` fields in each concept node are already structured as executable exercises. An assessment workflow would generate graded problem sets from the graph, ordered by tier.

- **Multi-language books**: the `@language` parameter is plumbed through `build_concept_book.spl` (ISO 639-1 codes; mathematical notation is language- invariant), though only English books have been generated to date. Producing the same book in Chinese or French is a one-parameter change we have not yet exercised end-to-end.

8 Conclusion

Teaching and learning are two sides of the same coin - and the concept-graph is the coin. A teacher authors it; a learner navigates it. One artifact, one SPL pipeline, two roles.

Two authoring paths make this accessible at scale. Path A lets any domain expert produce a concept-book from scratch - authoring the concept-graph as a structured file - with zero domain-specific code. Path B extracts a concept-graph from an existing open-source textbook, turning static prose into a navigable, graph-guided companion. The same SPL pipeline serves both.

Twenty domains - spanning mathematics, physics, chemistry, biology, computing, natural languages, and music - demonstrate our unified approach. A student can now trace `real_number` → `derivative` → `velocity` → `schrodinger_equation` as a single path across Calculus, Classical Mechanics, and Quantum Mechanics. The same LEGO payoff shape pattern - a small primitive brick set plus one composition rule generating a much larger concept space - underlies the design of all twenty, and is already documented as an empirical result at full domain scale in prior work on Chinese characters, where 422 elemental characters compose into the ~6,000 characters in common use [Gong 2025]. This paper contributes the architecture that makes the pattern executable and machine-checkable across arbitrary domains; whether it holds as an empirical law beyond the domains we authored is a question we leave open to independent replication.

The contribution is not a better LLM prompt or a more capable model. It is a *representation*: textbook knowledge re-represented as a structured concept-graph - prerequisite-ordered, reducible to its primitives, and navigable by teacher and learner alike. The LLM contributes breadth; the two-radical discipline enforces depth; the concept-graph makes both computable. ConceptBook is the system that makes a textbook computable and generative.

9 References

Abu-Rasheed, H., et al. (2025). LLM-Assisted Knowledge Graph Completion for Curriculum and Domain Modelling in Personalized Higher Education Recommendations. arXiv:2501.12300

ACM Computing Surveys (2025). Prerequisite Relation Learning: A Survey and Outlook. `https://dl.acm.org/doi/full/10.1145/3733593`

ALSaad et al. (2018). Mining MOOC Lecture Transcripts to Construct Concept Dependency Graphs. Educational Data Mining (EDM).

Anderson, J. R., Corbett, A. T., Koedinger, K. R., & Pelletier, R. (1995). Cognitive tutors: Lessons learned. The Journal of the Learning Sciences, 4(2).

Christou, A., et al. (2025). An Ontology for Representing Curriculum and Learning Material. arXiv:2506.05751

Doignon, J.-P., & Falmagne, J.-C. (1985). Spaces for the assessment of knowledge. International Journal of Man–Machine Studies, 23(2).

Gong, W. G. (2025). A New Exploration into Chinese Characters: from Simplification to Deeper Understanding. arXiv:2502.19428

Gong, W. G. (2026a). Structured Prompt Language: Declarative Context Management for LLMs. arXiv:2602.21257

Gong, W. G. (2026b). SPL: Orchestrating Workflows with Declarative Deterministic–Probabilistic Composition. TMLR (under review).

Graesser, A. C., et al. (2004). AutoTutor: A tutor with dialogue in natural language. Behavior Research Methods, Instruments, & Computers, 36(2).

Gunasekar, S., et al. (2023). Textbooks Are All You Need. arXiv:2306.11644

Hagberg, A., Schult, D., & Swart, P. (2008). Exploring network structure, dynamics, and function using NetworkX. SciPy Proceedings.

IMS Global Learning Consortium (2003). IMS Learning Design Specification.

Khan Academy (2023). Khanmigo: AI-powered tutor and teaching assistant. `https://www.khanacademy.org/khan-labs`

Li, et al. (2015). Component-enhanced Chinese character embeddings. arXiv:1508.06669

Liang, H., et al. (2026). K12-KGraph: A Curriculum-Aligned Knowledge Graph for Benchmarking and Training Educational LLMs. arXiv:2605.09635

Liu, H., Ma, W., Yang, Y., & Carbonell, J. (2016). Learning Concept Graphs from Online Educational Data. Journal of Artificial Intelligence Research, 55, 1059–1090.

Loach, B., & Wang, J. (2016). Optimizing the learning order of Chinese characters using a novel topological sort algorithm. PLOS ONE, 11(8).

Pan, L., et al. (2017). Prerequisite relation learning for concepts in MOOCs. ACL.

Pei, J., et al. (2025). Conversational Education at Scale: A Multi-LLM Agent Workflow for Procedural Learning and Pedagogic Quality Assessment. arXiv:2507.05528

Piech, C., et al. (2015). Deep Knowledge Tracing. NeurIPS.

Roy, et al. (2019). Inferring Concept Prerequisite Relations from Online Educational Resources. https://aaai.org/papers/09589-inferring-concept-prerequisite-relations-from-online-educational-resources/

Talukdar, P. P., & Cohen, W. W. (2012). Crowdsourced comprehension: Predicting prerequisite structure in Wikipedia. BEA Workshop (NAACL).

van der Linden, W. J., & Hambleton, R. K. (Eds.) (1997). Handbook of Modern Item Response Theory. Springer.

VanLehn, K. (2011). The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. Educational Psychologist, 46(4).

vis.js contributors. vis-network: dynamic, browser-based visualization library. https://visjs.github.io/vis-network/

Yarbro, J. T., & Olney, A. M. (2021). WikiMorph: Learning to Decompose Words into Morphological Structures. https://files.eric.ed.gov/fulltext/ED618429.pdf

Appendix A: SPL Workflow Listings

A.1 `build_concept_book.spl` - Full Workflow Listing

The workflow is domain-agnostic: switching from Linear Algebra to Classical Mechanics to Chinese Characters requires changing `@domain_yaml` only. Switching content depth (introductory survey vs. graduate-level treatment) requires changing `@lvl` only. All other logic - graph load, teaching order, verifier dispatch, cache, book assembly - is driven entirely by the concept-graph content.

```
WORKFLOW build_concept_book
    INPUT:
        @domain_yaml TEXT DEFAULT 'linalg_graph.yaml',
        @target      TEXT DEFAULT 'spectral_theorem',
        @lvl         TEXT DEFAULT 'intro',
        @language    TEXT DEFAULT 'en',
        @output_dir  TEXT DEFAULT ""
    OUTPUT:
        @textbook TEXT
DO
    -- 1. Load graph, validate (acyclicity + reducibility), compute teaching order
    CALL setup_domain(@domain_yaml, @target, @payoff_weight) INTO @_
    CALL order_length(@domain_yaml) INTO @order_len

    -- 2. Generate and verify one section per concept in dependency order
    @_i := 0
    WHILE @_i < @order_len DO
        CALL order_item(@domain_yaml, @_i) INTO @concept

        CALL cache_get(@concept) INTO @section
        EVALUATE @section
            WHEN = "miss" THEN
                GENERATE write_section(@concept, ...) INTO @section

                -- Primitive budget: no more than @primitive_budget (default 2)
                --   new primitives per section
                CALL needs_primitive_refinement(...) INTO @_refine
                EVALUATE @_refine
                    WHEN = "yes" THEN
                        GENERATE refine_section(@section, \
                          f"{@primitive_budget} new primitives per section") INTO @section
                END

                -- Domain verifier: structural / sympy / sage / lean
                CALL verify_section(@section, @domain_yaml) INTO @check
                EVALUATE @check
                    WHEN contains("fail") THEN
                        GENERATE refine_section(@section, @check, @lvl_guide) INTO @section
                END
                CALL cache_put(@concept, @section) INTO @_
            ELSE
                LOGGING "cache HIT — 0 LLM calls" LEVEL INFO
        END
        @textbook := @textbook + @section
        @_i := @_i + 1
    END
```

```
    -- 3. Capstone payoff section
    CALL apps_list(@domain_yaml) INTO @apps
    GENERATE write_payoff(@target, @apps, @lvl_guide) INTO @capstone
    @textbook := @textbook + @capstone

    COMMIT @textbook
END
```

A.2 `answer_on_demand.spl` - Personalized Learning-Gap Workflow

`answer_on_demand.spl` reuses the same tool library as `build_concept_book.spl` (Appendix A.1) but replaces the domain-wide teaching order with a personalized learning gap: it resolves a natural-language question to a target concept, computes the prerequisite concepts a specific learner (`@learner_state`) still needs via `setup_answer_path`, and generates only those missing sections before the target capstone. No new architecture is required - the same primitive-budget check, verifier dispatch, and section-generation prompts from Appendix A.1 are called unchanged (elided below; full source in the project repository).

```
WORKFLOW answer_on_demand
    INPUT:
        @question      TEXT DEFAULT 'Why does diagonalization work?',
        @domain_yaml   TEXT DEFAULT 'linalg_graph.yaml',
        @lvl           TEXT DEFAULT 'college',
        @learner_state TEXT DEFAULT '[]'
    OUTPUT:
        @lesson TEXT
DO
    -- 1. Resolve the NL question to a concept node (LLM); assert it exists
    CALL concept_names_list(@domain_yaml) INTO @concept_list
    GENERATE resolve_target(@question, @concept_list) INTO @target
    ASSERT in_graph(@domain_yaml, @target)
        OTHERWISE GENERATE resolve_target(@question, @concept_list) INTO @target

    -- 2. Personalized learning gap: prerequisites this learner still needs
    --    before reaching @target, in dependency order (cf. order_length/
    --    order_item over the whole domain in build_concept_book.spl)
    CALL setup_answer_path(@domain_yaml, @target, @learner_state) INTO @order_len

    -- 3. Generate and verify only the gap concepts (primitive budget,
    --    domain verifier, refine loop - identical checks to Appendix A.1)
    @lesson := ""
    @_i := 0
    WHILE @_i < @order_len DO
        CALL answer_path_item(@domain_yaml, @_i) INTO @concept
        GENERATE write_section(@concept, ...) INTO @section
        ... -- primitive budget check + verify_section + refine, as in A.1
        @lesson := @lesson + "\n\n---\n\n" + @section
        @_i := @_i + 1
    END

    -- 4. Target concept as the capstone
    GENERATE write_section(@target, ...) INTO @capstone
    @lesson := @lesson + "\n\n---\n\n" + @capstone
```

```
    RETURN @lesson
END
```

Appendix B: Full Domain Catalog

Table B1. Twenty domains: two-radical structure.

Domain	First radical (FORM)	Second radical	Irreducibility theorem
English Morphology	5 roots (spect, port, …)	prefixes + suffixes	Roots alone cannot yield *inspection*
Calculus	function, limit (analysis)	derivative, integral (operations)	Analysis alone cannot yield the Fundamental Theorem
Classical Mechanics	position, time (kinematics)	mass, force (dynamics)	No kinematic quantity predicts F=ma
Chinese Characters	11 semantic pictograms	马 (phonetic lender)	Pictograms alone cannot yield phono-semantic compounds
Chemistry Elements	8 elements (H,C,N,…)	valence_electron	Element identity alone does not predict bonding
Music Theory	pitch, semitone (pitch space)	beat, halving (rhythm)	Pitch alone cannot yield rhythm structures
Linear Algebra	vectors, scalars (objects)	linear map, inner product (operations)	Vectors alone cannot yield the Spectral Theorem
Quantum Mechanics	state, observable (objects)	superposition, measurement (operations)	States alone cannot yield Born rule predictions
Biology	cell, molecule (structures)	replication, transcription (processes)	Structures alone cannot yield cell division
Molecular Biology	nucleotide, codon (structures)	transcription, translation (processes)	Structure alone cannot yield protein synthesis

Table B2. Twenty domains: primitive count, concept count, capstone, verifier family, and the LEGO payoff.

Symbolic / Algorithmic - exact computation, formal verification

Domain	Primitives	Concepts	Capstone	Verifier	LEGO Payoff
Calculus	4	~20	Fundamental Theorem	SymPy	4 primitives → physics, ML, economics, engineering
Linear Algebra	5	25	Spectral Theorem	SymPy	5 primitives → ML, quantum, signal processing
Introductory Geometry	5	19	Trigonometric Ratios	SymPy / Sage	5 primitives → navigation, architecture, graphics
Classical Mechanics	4	19	Normal Modes	SymPy / Sage	4 primitives → robotics, orbital mechanics, quantum bridge
Quantum Mechanics	5	~18	Path Integral	SymPy / Sage	5 primitives → QFT, statistical mechanics, quantum gravity
SageMath	4	18	Experimental Mathematics	Sage / SymPy	4 primitives → CAS-aided mathematics
Lean Theorem Proving	4	19	AI-Assisted Proving	Lean / SymPy	4 primitives → formal proof, AI mathematics

Domain	Primitives	Concepts	Capstone	Verifier	LEGO Payoff
Algorithms	5	~18	Dynamic Programming	Structural / SymPy	5 primitives → algorithm design across domains
Data Structures	4	~16	Graph Algorithms	Structural	4 primitives → any data-intensive system
SQL & Relational Algebra	5	~16	Execution Plan Tuning	Structural / SymPy	5 primitives → any query optimization problem

Network-Driven / Balanced - hierarchical composition, conservation laws

Domain	Primitives	Concepts	Capstone	Verifier	LEGO Payoff
Chemistry Elements	9	14	Stoichiometry	Structural	9 bricks → every formula on every label
Python for Science	4	19	Reproducible Science	NumPy/ SymPy/ SciPy/ sklearn	4 primitives → full scientific Python stack
Physics Ch1: Nature of Science	4	~12	Measurement & Error Analysis	Structural/ SymPy	4 primitives → experimental foundations
Physics Ch2: Kinematics	4	~14	Projectile Motion	SymPy	4 primitives → all 1D/2D motion problems
Biology	5	~16	Cell Division	Structural	5 primitives → genetics, development, evolution
Medicine	5	~18	Systems Diagnosis	Structural	5 primitives → clinical reasoning frameworks
Molecular Biology	5	~16	Protein Synthesis Regulation	Structural	5 primitives → gene expression, biotechnology

Morphological / Structural - decomposition, composition rules, pattern recognition

Domain	Primitives	Concepts	Capstone	Verifier	LEGO Payoff
Chinese Characters	12	16	Phono-semantic Principle	Structural	422 elemental characters → ~6,000 characters
English Morphology	13	18	Morphological Decoding	Structural	~220 morphemes → most academic vocabulary
Music Theory	4	17	Four-Chord Loop	Structural	4 primitives → most popular music

Appendix C: ConceptBook Web App Interface

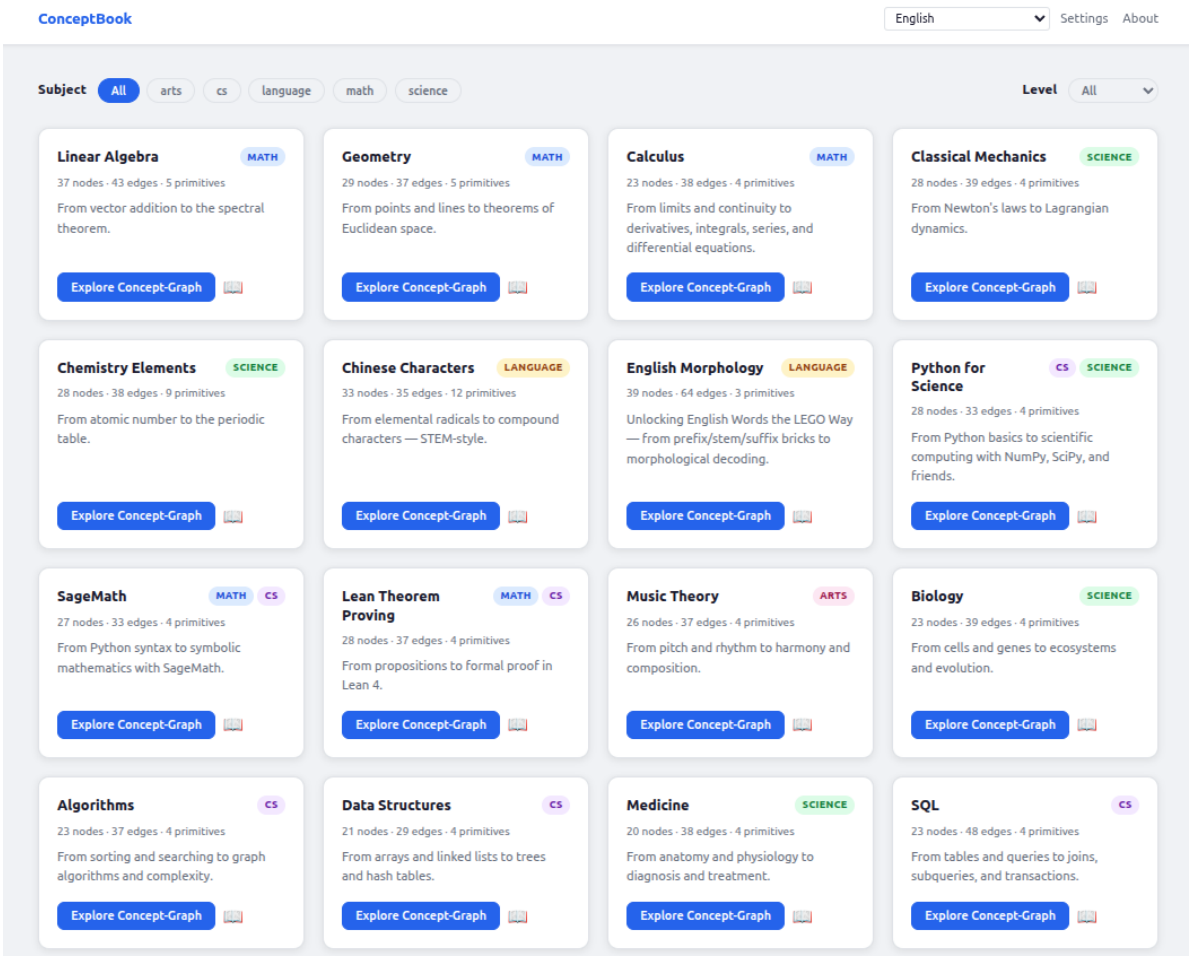

Figure 4: ConceptBook web app domain grid. The home page displays twenty concept-graph domains organized by subject tag (math, science, cs, language, arts), showing domain name, node/edge counts, primitive count, and a brief description. Each domain card links to an interactive navigator view where learners can explore prerequisite paths and generate personalized concept-books. This unified interface demonstrates the schema's horizontal scalability across heterogeneous domains.

The ConceptBook system is deployed as a web application combining a Vite + Vanilla JS frontend (served via GitHub Pages) with a FastAPI backend for on-demand content generation. The domain grid (Figure 4) provides an entry point to twenty concept-graphs. Users can select a domain, choose a target concept and content level, and stream the generated concept-book in real-time via Server-Sent Events. The same vis.js navigator that visualizes the domain graph (Figure 3) serves as the learner's navigation interface for exploring prerequisite paths and identifying gaps.

Appendix D: Generated Concept-Book Samples

Sample generated concept-books across representative domains are available in the open-source repository on GitHub. The following links provide complete, validated concept-books (intro, core, college, and research levels where applicable) generated by the SPL pipeline and reviewed by domain experts.

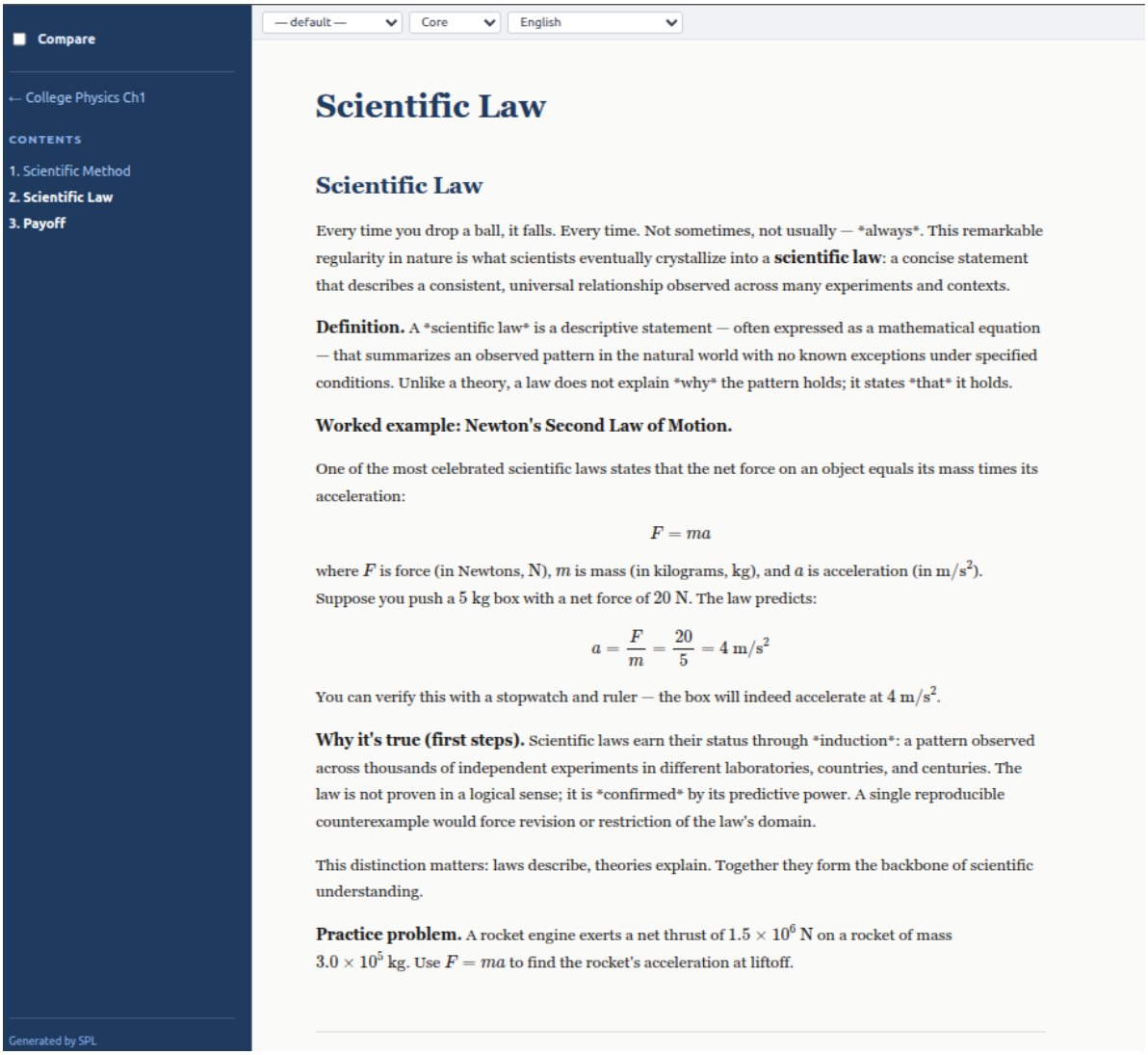

Figure 5: ConceptBook reader view: a generated section ("Scientific Law", College Physics Ch1, `core` level) showing the law statement, worked example, and practice problem produced by the SPL pipeline, alongside the domain/level/language selectors and section navigator.

D.1 Classical Mechanics

Target concept: `normal_modes` (capstone). Full concept-book: `https://github.com/digital-duck/concept-book/tree/main/public/domains/classical_mechanics/output/college.en/sonnet/html`

This domain demonstrates the physics foundation set (position, time, mass, force) composing toward wave phenomena. The generated treatment covers velocity, momentum, energy conservation, eigenvalues, superposition, and culminates in normal modes as the capstone. The structural validator confirms all energy expressions reduce to kinetic + potential forms; SymPy verifies worked examples.

D.2 Chinese Characters

Target concept: `phono_semantic_principle` (capstone). Full concept-book: `https://github.com/digital-duck/concept-book/tree/main/public/domains/chinese_characters/output/core.en/sonnet/html`

This domain applies the framework to the ZiNets analysis of Chinese characters: 422 elemental characters (元字) partition into semantic pictograms (FORM) and phonetic lenders (SOUND). The structural verifier confirms that every composite character in the corpus decomposes into the declared primitives. Generation includes character etymology and modern usage examples.

D.3 Computer Science: Algorithms

Target concept: `dynamic_programming` (capstone). Full concept-book: `https://github.com/digital-duck/concept-book/tree/main/public/domains/algorithms/output/college.en/sonnet/html`

This domain builds from primitives (input, recursion, state) toward the capstone dynamic programming. The teaching order places recursion and memoization as prerequisites to optimal substructure and the DP recurrence relation. All worked examples are executable Python with test cases.

D.4 Mathematics: Calculus

Target concept: `fundamental_theorem` (capstone). Full concept-book: `https://github.com/digital-duck/concept-book/tree/main/public/domains/calculus/output/college.en/sonnet/html`

This domain derives from limits and continuity (FORM) toward derivatives and integrals (SOUND), with the Fundamental Theorem as capstone. SymPy symbolic verification confirms derivative and integral identities; worked examples include inverse-operation proofs.

D.5 Morphological Language: English

Target concept: `morphological_decoding` (capstone). Full concept-book: `https://github.com/digital-duck/concept-book/tree/main/public/domains/english_morphology/output/intro.en/sonnet/html`

This domain applies compositional structure to English: 13 roots (spect-, port-, -cede-, etc.) compose with affixes (in-, -tion, re-, -able) to generate academic vocabulary. The structural verifier decomposes derived words (inspection = in + spect + -ion, transportation = trans + port + -tion) into declared primitives. Demonstrates the framework's application to naturally compositional languages beyond symbolic mathematics.